IS 1210
Database Systems

Lecture 8 – Structured Query Language (SQL)

MVPT Lakshika,

Lecturer (Probationary),

Department of Information Systems Engineering, UCSC.

Contact: tlv@ucsc.cmb.ac.lk

Outline of this session…

1. Indexes
2. Views

1. INDEXES

DDL Continues …

CREATE SCHEMA CREATE DOMAIN CREATE TABLE CREATE VIEW CREATE INDEX	ALTER DOMAIN ALTER TABLE	DROP SCHEMA DROP DOMAIN DROP TABLE DROP VIEW DROP INDEX

Index in books

Index

A KIND OF relationship 820
A-PART-OF (APO) relationship 824
A-PART-OF hierarchy 824
aborted transaction 574, 746
Abstract Data Types 815, 820
  in Postgres 943–4
abstraction in object-orientation 814
access control
  as security 547–50
  in SQL 189–94
    granting privileges 191–2
    revoking privileges 192–4
Access database system 25
access delay in DDBMS 703
access methods 1270
  in OODBMS 835
access plan 667
ACID properties of transactions 575–6, 758
action query 200, 215–23
  in SQL 139
ALL in SQL 138–9
allocation-based persistence 869
allow zero length property in Access 232
alternate key 79, 353
  in conceptual design 451–2, 1327
American National Standards Institute (ANSI) 25, 34
Analytical Queries per Minute (AQM) 1206
And operator in QBE 202
ANY in SQL 138–9
ANY/SOME in SQL 138
append query 220–3
Apple Talk 702
application developers 23
application development tools in data warehousing 1160–1
application gateway 564
application generators 43
  in conceptual design 450–1, 1327
  constraints in logical design 475
attributes 15, 20, 33, 44, 69, 72, 350–4
  composite 351
  in conceptual design 447–50, 1327
  definitions in SQL 948
  derived 352
    in conceptual database design 448
    in physical database design 499
  distributing 529
  inheritance in EER model 374
  multi-valued 352
    in conceptual design 448
    in logical design 471–2
  of objects 905
  in object-orientation 815–16
  on relationships 355–6
    representation of 356
  representation of 353–4
  simple 351

What is an Index?

A structured list of important terms or topics that helps readers quickly locate information within a book.

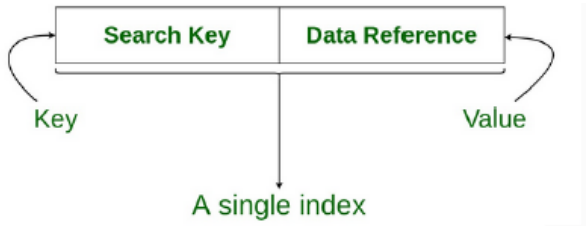
Provides,

✓ fast access
✓ topic-based navigation
✓ no need to read sequentially

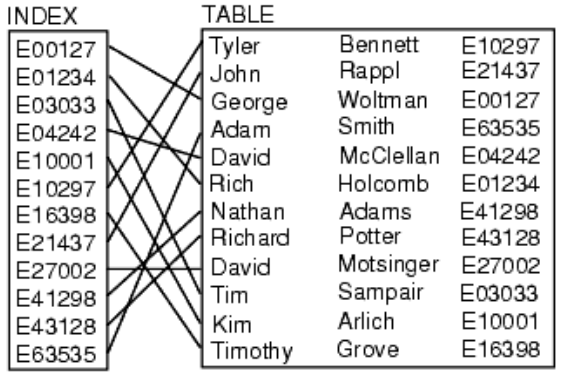
Introduction

- An index is a data structure that allows the DBMS to locate particular records in a file more quickly and thereby speed response to user queries, **especially in large tables.**
  - Provides accelerated access to the rows of a table based on the values of one or more columns.
  - An index significantly improves the performance of a query. You may not notice this improvement with small tables, but it can be significant for large tables.
  - An index minimize the number of disk accesses required when a query is processed.
  - Indexes are usually created to satisfy particular search criteria after the table has been in use for some time and has grown in size.
- A database table can have one or more indexes associated with it.

Index structure



- An index structure is associated with a search key and data reference.
  - The first column is the **Search key** that contains a copy of the primary key or candidate key of the table. These values are stored in sorted order so that the corresponding data can be accessed quickly (The data may or may not be stored in sorted order of the logical record).
  - The second column is the **Data Reference or Pointer,** which contains a set of pointers holding the address of the logical record in the disk block where that particular key value can be found.



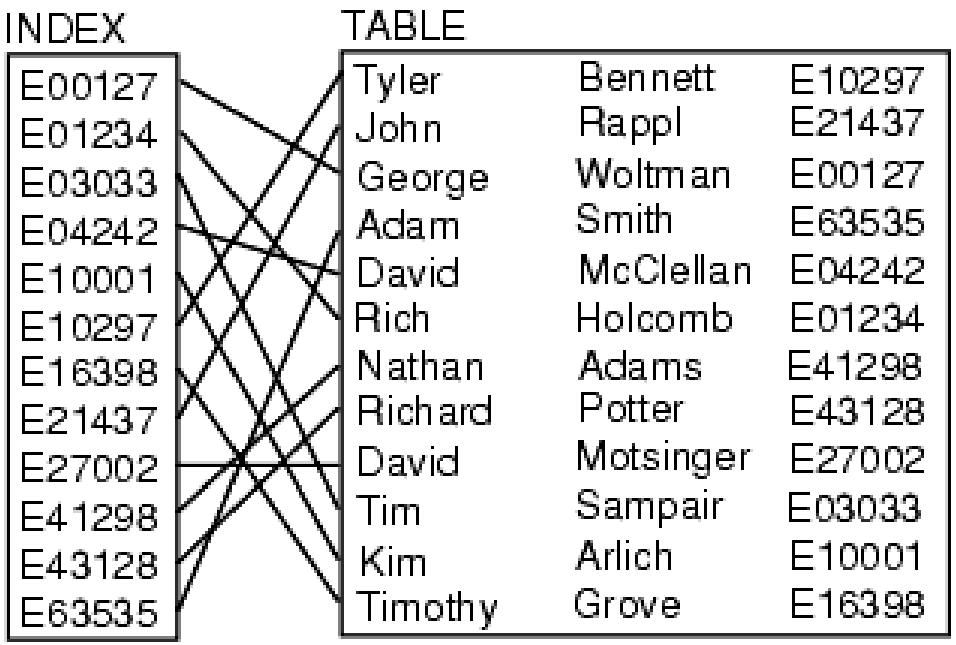
- The file containing the logical records is called the data file, and the file containing the index records is called the index file.
- The values in the index file are ordered according to the *indexing field*, which is usually based on a single attribute.
  - An index is defined by a field expression that you specify when you create the index.

Index structure

Example 1: Consider the single field name, emp_id.

e.g., An index created on the emp_id field contains a sorted list of the employee ID values in the table.

Each value in the list is accompanied by references to the rows that contain that value.



- Consider the following WHERE clause:

  WHERE EMP_id = 'E10001'

  -Without an index, the server must search the entire database table to find those rows having an employee ID of E10001.

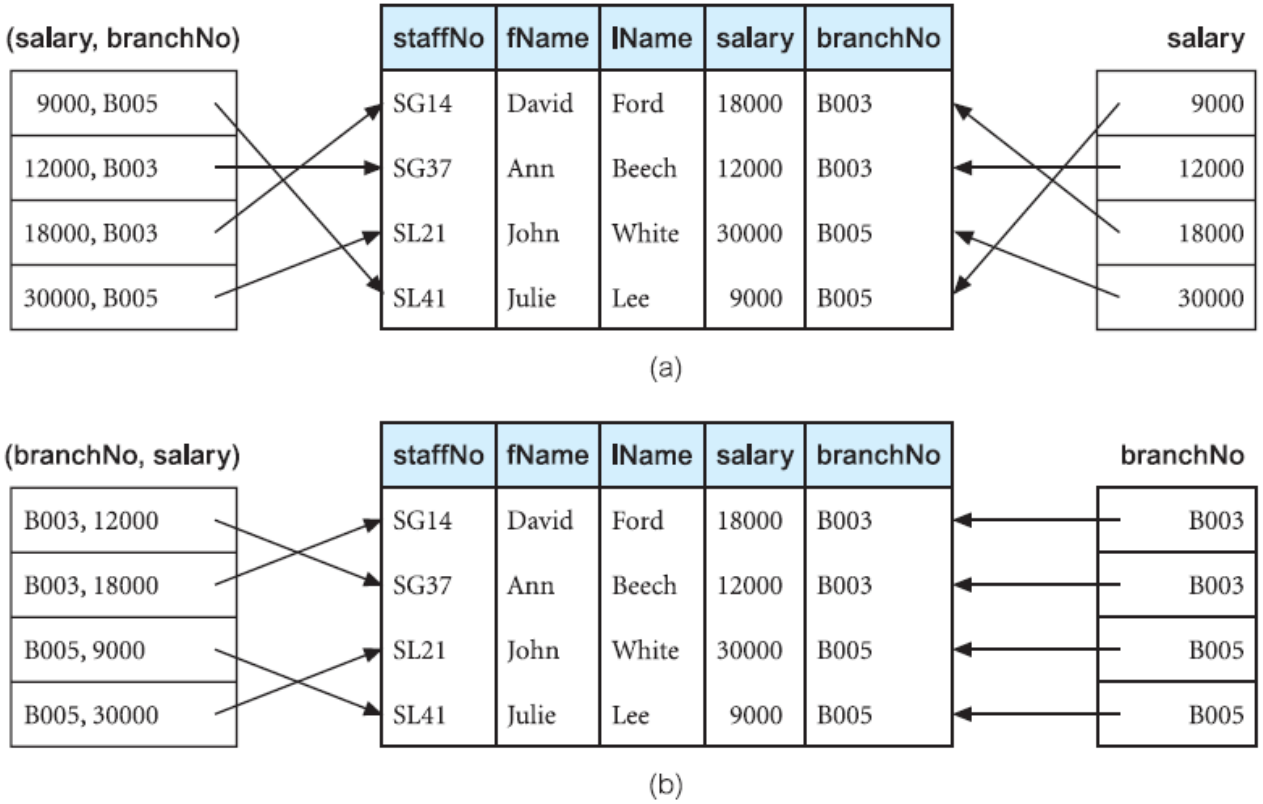
  - With an index, the server can quickly find those rows.

Example

- The search key for an index can consist of one or more fields.

Example 2: Four dense indexes on the (reduced) Staff table

- Based on the salary column
- Based on the branchNo column
- Based on the composite index (salary, branchNo)
- Based on the composite index (branchNo, salary)



Index structure

- While indexes are not strictly necessary to use the DBMS, they can have a significant impact on performance.
  - Having too many indexes can slow down the performance of some inserts, updates, and deletes when the driver has to maintain the indexes as well as the database tables.
  - Consume additional disk space.
  - Since indexes may be updated by the system every time the underlying tables are updated, additional overheads may be incurred.

Types of Index

There are different types of index, the main ones being:

1. **Primary index:** The data file is sequentially ordered by an ordering key field, and the indexing field is built on the ordering key field, which is guaranteed to have a unique value in each record.
2. **Clustering index:** The data file is sequentially ordered on a non-key field, and the indexing field is built on this non-key field, so that there can be more than one record corresponding to a value of the indexing field. The non-key field is called a *clustering attribute*.
3. **Secondary index:** An index that is defined on a non-ordering field of the data file.

A file can have *at most* one primary index or one clustering index, and can also have several secondary indexes.

In addition, an index can be **sparse** or **dense**: a sparse index has an index record for only some of the search key values in the file (Usually, it only points to the first record of each disk block); a dense index has an index record for every search key value in the file.

Create Index

- The creation of indexes is *not* standard SQL.
- However, most dialects support at least the following capabilities:

```
CREATE [UNIQUE] INDEX IndexName
ON TableName (columnName [ASC | DESC] [, . . . ])
```

- The specified columns constitute the index key and should be listed in major to minor order.
- If the UNIQUE clause is used, the uniqueness of the indexed column or combination of columns will be enforced by the DBMS.
- This is certainly required for the primary key and possibly for other columns as well (example, for alternate keys).

Create Index

- For the Staff and PropertyForRent tables, we may want to create at least the following indexes:

  CREATE UNIQUE INDEX StaffNoInd **ON** Staff (staffNo);

  CREATE UNIQUE INDEX PropertyNoInd **ON** PropertyForRent (propertyNo);

- For each column, specify that the order is ascending (ASC) or descending (DESC).
- For example, if we create an index on the PropertyForRent table as:

  CREATE INDEX RentInd **ON** PropertyForRent (city, rent);

then an index called RentInd is created for the PropertyForRent table. Entries will be in alphabetical order by city and then by rent within each city.

Create Index

- Indexes can be created only on base tables, not on views.
- When to create indexes?
  - Indexes can be created at any time.
  - A problem arises when attempting to create a UNIQUE index on a table that already contains data because the values in the indexed column(s) may include duplicate entries.
  - Therefore, it is good practice to create UNIQUE index, at least for primary key columns, when the base table is created, and the DBMS does not automatically enforce primary key uniqueness.

Create Index

Activity

Identify a clustering index for the PropertyForRent table. Write the SQL statement to create the index.

CREATE CLUSTERED INDEX idx_property_city **ON** PropertyForRent (city);

Identify secondary indexes for the staff and client tables. Write the SQL statements to create the index.

CREATE INDEX idx_staff_salary **ON** Staff (salary);

CREATE INDEX idx_client_lname **ON** Client (lName);

Improving Record Selection Performance

- For indexes to improve the performance of selections, the index expression must match the selection condition exactly.

Example 3: *If you have created an index whose expression is last_name, the following SELECT statement uses the index:*

```
SELECT * FROM emp WHERE last_name = 'Smith';
```

- However, the below SELECT statement does not use the index:

```
SELECT * FROM emp WHERE UPPER(last_name) = 'SMITH' ;
```

  - The WHERE clause contains UPPER(LAST_NAME), which does not match the index expression LAST_NAME.
  - If you plan to use the UPPER function in all your SELECT statements and your database supports indexes on expressions, then you should define an index using the expression UPPER(LAST_NAME).

Indexing multiple fields

- If you often use WHERE clauses that involve more than one field, you may want to build an index containing multiple fields.
- Consider the following Where clause:

  e.g., WHERE last_name = 'Smith' AND first_name = 'Thomas'

- For this condition, the optimal index field expression is LAST_NAME, FIRST_NAME.
- This creates a concatenated index.
  - Concatenated indexes can also be used for WHERE clauses that contain only the first of two concatenated fields.
  - The LAST_NAME, FIRST_NAME index also improves the performance of the following WHERE clause (even though no first name value is specified): last_name = 'Smith'

Removing an Index (DROP INDEX)

- If you create an index for a base table and later decide that it is no longer needed, you can use the **DROP INDEX** statement to remove the index from the database.

  DROP INDEX index_name;

  DROP INDEX index_name **ON** table_name; (MySQL)

- The following statement will remove the index created in the previous example:

  DROP INDEX RentInd;

- To check the existing index on a table (In MySQL):

  SHOW INDEX FROM table_name;

2. VIEWS

Introduction

What is a View?

- A view is a virtual relation that does not necessarily exist in the database but can be produced on request by a particular user at the time of the request.
  - The dynamic result of one or more relational operations operating on the base relations to produce another relation.
  - To the database user, a view appears just like a real table, with a set of named columns and rows of data.
  - However, unlike a base table, a view does not necessarily exist in the database as a stored set of data values.
  - Instead, a view is defined as a query on one or more base tables or views.
  - The DBMS stores the definition of the view in the database.

Creating a View

- The format of the CREATE VIEW statement is:

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]
AS subselect [WITH [CASCADED | LOCAL] CHECK OPTION]
```

- A name may optionally be assigned to each column in the view. Specified names, if there is any ambiguity in the name of a column.

  Ex: when a *subselect* includes calculated columns, and the **AS** subclause has not been used to name such columns, two columns with identical names as the result of a join operation

- If a list of column names is specified,

  - It must have the same number of items as the number of columns produced by the *subselect*.

- If the list of column names is omitted,

  - Each column in the view takes the name of the corresponding column in the *subselect* statement.

Creating a View

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]
AS subselect [WITH [CASCADED | LOCAL] CHECK OPTION]
```

- The **subselect** is known as the defining query.
  - If WITH CHECK OPTION is specified, SQL ensures that if a row fails to satisfy the WHERE clause of the defining query of the view, it is not added to the underlying base table of the view.
- It should be noted that to create a view successfully, you must have SELECT privilege on all the tables referenced in the *subselect* and USAGE privilege on any domains used in referenced columns.
- Although all views are created in the same way, in practice, different types of views are used for different purposes.

Create a horizontal view

- A horizontal view restricts a user's access to selected rows of one or more tables.

Example 1: *Create a view so that the manager at branch B003 can see only the details for staff who work in his or her branch office.*

```
CREATE VIEW Manager3Staff AS
SELECT *
FROM Staff
WHERE branchNo = 'B003';
```

subselect

- The view *Manager3Staff* includes the same column names as the Staff table but contains only those rows where the branch number is B003.

Create a horizontal view

- How to execute the statement?

  SELECT * **FROM** Manager3Staff;

Result:

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

- To ensure that the branch manager can see only these rows,
  - The manager should not be given access to the base table Staff.
  - Instead, the manager should be given access permission to view Manager3Staff.
- This, in effect, gives the branch manager a customized view of the Staff table, showing only the staff at his or her own branch.

Create a vertical view

- A vertical view restricts a user's access to selected columns of one or more tables.

Example 2: *Create a view of staff details at branch B003 that excludes salary information, so that only managers can access salary information for staff in their branch.*

Approach 1:

```
CREATE VIEW Staff3 AS
SELECT staffNo, fName, lName, position, sex
FROM Staff
WHERE branchNo = 'B003';
```

- Use the Manager3Staff view instead of the Staff table.

Approach 2:

```
CREATE VIEW Staff3 AS
SELECT staffNo, fName, lName, position, sex
FROM Manager3Staff;
```

Result:

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

Create a vertical view

- Both approaches create a view called Staff3 with the same columns as the Staff table, but excluding the *salary, DOB*, and *branchNo* columns.

Result:

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

- To ensure that only the branch manager can see the salary details, staff at branch B003 should not be given access to the base table Staff or the view Manager3Staff.
- Instead, they should be given access permission to view Staff3, thereby denying them access to sensitive salary data.
- Vertical views are commonly used where various users or groups of users use the data stored in a table. They provide a private table for these users composed only of the columns they need.

Grouped and Joined views

Example 3: *Create a view of staff who manage properties for rent, which includes the branch number they work at, their staff number, and the number of properties they manage.*

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt) AS
SELECT s.branchNo, s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;
```

Result:

branchNo	staffNo	cnt
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

- Rename the columns in the definition of the view because of the use of the unqualified aggregate function COUNT in the *subselect*.

Creating Views

Activity

Create a view that allows managers to see which staff members are managing which properties and the rent those properties generate.

```
CREATE VIEW StaffPropertyView AS
SELECT s.branchNo, s.fName, s.lName, p.propertyNo, p.city, p.rent
FROM Staff s JOIN PropertyForRent p ON s.staffNo = p.staffNo;
```

List all the information in the created view.

```
SELECT * FROM StaffPropertyView;
```

List information related to branch B003.

```
SELECT * FROM StaffPropertyView WHERE branchNo ='B003';
```

Removing a View

- A view is removed from the database with the **DROP VIEW** statement:

```
DROP VIEW ViewName [RESTRICT | CASCADE]
```

- DROP VIEW causes the definition of the view to be deleted from the database.

Example 4: *Remove the Manager3Staff view using the statement:*

DROP VIEW Manager3Staff;

- If CASCADE is specified,
  - DROP VIEW deletes all related dependent objects (all objects that reference the view). This means that DROP VIEW also deletes any views that are defined on the view being dropped.
- If RESTRICT is specified (default setting),
  - Any other objects that depend for their existence on the continued existence of the view being dropped, the command is rejected.

Handling views

- Two primary ways the database engine handles the execution and storage of a view: **View Resolution** and **View Materialization**.
- When the DBMS encounters a reference to a view,
  - **Approach 1: view resolution**

    Look up this definition and translate the request into an equivalent request against the source tables of the view, and then perform the equivalent request.
  - **Approach 2: view materialization**

    Stores the view as a temporary or physical table in the database and maintains the currency of the view as the underlying base tables are updated.

View Resolution

Example 5: *Counts the number of properties managed by each member of staff at branch office B003.*

```
SELECT staffNo, cnt
FROM StaffPropCnt
WHERE branchNo = 'B003'
ORDER BY staffNo;
```

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt) AS
SELECT s.branchNo, s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;
```

- This query is based on the StaffPropCnt view.
- View resolution merges the above query with the StaffPropCnt view as follows:

1). The view column names in the SELECT list are translated into their corresponding column names in the defining query.

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt
```

View Resolution (Cont…)

```
SELECT staffNo, cnt
FROM StaffPropCnt
WHERE branchNo = 'B003'
ORDER BY staffNo;
```

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt)
        AS SELECT s.branchNo, s.staffNo, COUNT(*)
        FROM Staff s, PropertyForRent p
        WHERE s.staffNo = p.staffNo
        GROUP BY s.branchNo, s.staffNo;
```

2). View names in the FROM clause are replaced with the corresponding FROM lists of the defining query:

FROM Staff s, PropertyForRent p

3). The WHERE clause from the user query is combined with the WHERE clause of the defining query using the logical operator **AND**.

WHERE s.staffNo = p.staffNo **AND** branchNo = 'B003'

4). The GROUP BY and HAVING clauses are copied from the defining query.

GROUP BY s.branchNo, s.staffNo

View Resolution (Cont…)

```
SELECT staffNo, cnt
FROM StaffPropCnt
WHERE branchNo = 'B003'
ORDER BY staffNo;
```

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt)
        AS SELECT s.branchNo, s.staffNo, COUNT(*)
        FROM Staff s, PropertyForRent p
        WHERE s.staffNo = p.staffNo
        GROUP BY s.branchNo, s.staffNo;
```

5). Finally, the ORDER BY clause is copied from the user query with the view column name translated into the defining query column name:

ORDER BY s.staffNo

6). The final merged query becomes:

```
SELECT s.staffNo AS staffNo, COUNT(*) AS cnt
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo AND branchNo = 'B003'
GROUP BY s.branchNo, s.staffNo
ORDER BY s.staffNo;
```

Result table after view resolution:

staffNo	cnt
SG14	1
SG37	2

Restrictions on Views

- The ISO standard imposes several important restrictions on the creation and use of views, although there is considerable variation among dialects.
- If a column in the view is based on an aggregate function, then the column may appear only in **SELECT** and **ORDER BY** clauses of queries that access the view.
- In particular, such a column may not be used in a WHERE clause and may not be an argument to an aggregate function in any query based on the view.

Example 6: *Consider the view StaffPropCnt, which has a column cnt based on the aggregate function COUNT.*

```
SELECT COUNT(cnt)
FROM StaffPropCnt;
```

This query would fail because it uses an aggregate function on the column cnt, which is itself based on an aggregate function.

Restrictions on Views

Example 6 (Cont…):

```
SELECT *
FROM StaffPropCnt
WHERE cnt > 2;
```

This query would also fail because it uses the view column, cnt, derived from an aggregate function in a WHERE clause.

- A grouped view may never be joined with a base table or a view.

Example: The StaffPropCnt view is a grouped view, so that any attempt to join this view with another table or view fails.

```
CREATE VIEW StaffPropCnt (branchNo, staffNo, cnt)
        AS SELECT s.branchNo, s.staffNo, COUNT(*)
        FROM Staff s, PropertyForRent p
        WHERE s.staffNo = p.staffNo
        GROUP BY s.branchNo, s.staffNo;
```

View updatability

- All updates to a base table are immediately reflected in all views that encompass that base table.
- Similarly, we may expect that if a view is updated, then the base table(s) will reflect that change.
- Consider the view StaffPropCnt. What would happen if you tried to insert a record that showed that at branch B003, staff member SG5 manages two properties?

```
INSERT INTO StaffPropCnt
VALUES ('B003', 'SG5', 2);
```

  - You have to insert two records into the PropertyForRent table showing which properties staff member SG5 manages.
  - But, you do not know which properties they are (You do not know the corresponding primary key values for the PropertyForRent table)

View updatability

If we change the definition of the view and replace the count with the actual property numbers?

```
CREATE VIEW StaffPropList (branchNo, staffNo, propertyNo)
        AS SELECT s.branchNo, s.staffNo, p.propertyNo
        FROM Staff s, PropertyForRent p
        WHERE s.staffNo = p.staffNo;
```

Then try to insert the record:

```
INSERT INTO StaffPropList VALUES ('B003', 'SG5', 'PG19');
```

- Still, there is a problem with this insertion.
  - The definition of the PropertyForRent table (All columns except *postcode* and *staffNo* were not allowed to have *nulls*).
  - However, as the StaffPropList view excludes all columns from the PropertyForRent table except the property number, we have no way of providing the remaining non-null columns with values.

For a view to be updatable, the DBMS must be able to trace any row view or column back to its row or column in the source table.

View updatability

- Following the ISO standard, a view is updatable if and only if:
  - DISTINCT is not specified (duplicate rows must not be eliminated from the query results)
  - Every element in the SELECT list of the defining query is a column name (rather than a constant, expression, or aggregate function), and no column name appears more than once.
  - The FROM clause specifies only one table (view must have a single source table for which the user has the required privileges). If the source table is itself a view, then that view must satisfy these conditions. This, therefore, **excludes** any views based on a join, union (UNION), intersection (INTERSECT), or difference (EXCEPT).
  - The WHERE clause does not include any nested SELECTs that reference the table in the FROM clause.
  - There is no GROUP BY or HAVING clause in the defining query.
- In addition, every row that is added through the view must not violate the integrity constraints of the base table.

Example: if a new row is added through a view, columns that are not included in the view are set to null, but this must not violate a NOT NULL integrity constraint in the base table.

Activity

The *DreamHome* management needs a report that lists every property currently for rent, along with the name of the staff member who manages it. Create a view named PropertyManagementReport to show: PropertyID, Street, City, and Manager Name (the full name of the staff member managing that property)

This data is split between the PropertyForRent and Staff tables.

```
CREATE VIEW PropertyManagementReport AS
SELECT p.propertyNo AS PropertyID, p.street AS Street, p.city AS City, CONCAT(s.fName, ' ', s.lName) AS ManagerName
FROM PropertyForRent p JOIN Staff s ON p.staffNo = s.staffNo;

CREATE VIEW PropertyManagementReport AS
SELECT propertyNo AS PropertyID, street AS Street, city AS City, CONCAT(fName, ' ', lName) AS ManagerName
FROM PropertyForRent JOIN Staff USING (staffNo);

CREATE VIEW PropertyManagementReport AS
SELECT propertyNo AS PropertyID, street AS Street, city AS City, CONCAT(fName, ' ', lName) AS ManagerName
FROM PropertyForRent NATURAL JOIN Staff;
```

WITH CHECK OPTION

- Rows exist in a view because they satisfy the WHERE condition of the defining query.
- If a row is altered such that it no longer satisfies this condition, then it will disappear from the view.
- Similarly, new rows will appear within the view when an insert or update on the view causes them to satisfy the WHERE condition.
- The rows that enter or leave a view are called **migrating rows**.
- The WITH CHECK OPTION clause of the CREATE VIEW statement prohibits a row from migrating out of the view.

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]
AS subselect [WITH [CASCADED | LOCAL] CHECK OPTION]
```

WITH CHECK OPTION

- The optional qualifiers LOCAL/CASCADED are applicable to view hierarchies (a view that is derived from another view).
  - If the **WITH CASCADED CHECK OPTION** is specified, then any row insert or update on this view (and on any view directly or indirectly defined on this view) must not cause the row to disappear from the view.
  - If **WITH LOCAL CHECK OPTION** is specified, then any row insert or update on this view, (and on any view directly or indirectly defined on this view), must not cause the row to disappear from the view, unless the row also disappears from the underlying derived view/table.
- This feature is so useful that it can make working with views more attractive than working with the base tables.
- When an INSERT or UPDATE statement on the view violates the WHERE condition of the defining query, the operation is rejected.
- The WITH CHECK OPTION can be specified only for an updatable view.

WITH CHECK OPTION

Example 6: Consider the *Manager3Staff* view created.

```
CREATE VIEW Manager3Staff AS
SELECT *
FROM Staff
WHERE branchNo = 'B003'
WITH CHECK OPTION;
```

Data for view Manager3Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Consider an attempt to <u>update</u> the branch number of one of the rows from B003 to B005.

```
UPDATE Manager3Staff
SET branchNo = 'B005'
WHERE staffNo = 'SG37';
```

- The WITH CHECK OPTION clause in the definition of the view prevents this from happening, as this would cause the row to migrate from this horizontal view.

WITH CHECK OPTION

Example 6: Consider the *Manager3Staff* view created.

```
CREATE VIEW Manager3Staff AS
SELECT *
FROM Staff
WHERE branchNo = 'B003'
WITH CHECK OPTION;
```

Data for view Manager3Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Consider an attempt to <u>insert</u> the following row through the view:

```
INSERT INTO Manager3Staff
VALUES('SL15', 'Mary', 'Black', 'Assistant', 'F', '1967-06-21', 8000, 'B002');
```

- The WITH CHECK OPTION would prevent the row from being inserted into the underlying Staff table and immediately disappearing from this view (as branch B002 is not part of the view).

WITH CHECK OPTION

Now consider the situation where Manager3Staff is defined not on Staff directly but on another view of Staff:

```
CREATE VIEW LowSalary AS
SELECT *
FROM Staff
WHERE salary > 9000;
```

```
CREATE VIEW HighSalary AS
SELECT *
FROM LowSalary
WHERE salary > 10000
WITH LOCAL CHECK OPTION;
```

```
CREATE VIEW Manager3Staff AS
SELECT *
FROM HighSalary
WHERE branchNo = 'B003';
```

Try the following update on Manager3Staff:

UPDATE Manager3Staff **SET** salary = 9500 **WHERE** staffNo = 'SG37';

- This update would fail. Although the update would cause the row to disappear from the view HighSalary, the row would not disappear from the table LowSalary that HighSalary is derived.

WITH CHECK OPTION

- However, if instead the update tried to set the salary to 8000, then the update would succeed as the row would no longer be part of LowSalary.
- Alternatively, if the view HighSalary had specified WITH CASCADED CHECK OPTION, then setting the salary to either 9500 or 8000 would be rejected because the row would disappear from HighSalary.
- Therefore, to ensure that anomalies like this do not arise, each view should normally be created using the **WITH CASCADED CHECK OPTION** (default option).

Advantages of views

- **Data independence:** A view can present a consistent, unchanging picture of the structure of the database, even if the underlying source tables are changed.
- **Currency:** Changes to any of the base tables in the defining query are immediately reflected in the view.
- **Improved security:** Each user can be given the privilege to access the database only through a small set of views that contain the data appropriate for that user (restricting and controlling each user's access to the database).
- **Reduced complexity:** A view can simplify queries by drawing data from several tables into a single table (transforming multi-table queries into single-table queries).
- **Convenience:** Views can provide greater convenience to users as users are presented with only that part of the database that they need to see (this reduces the complexity from the user's point of view).
- **Customization:** Views customize the appearance of the database, so that the same underlying base tables can be seen by different users in different ways.
- **Data integrity:** The WITH CHECK OPTION clause ensures that no row that fails to satisfy the WHERE clause of the defining query is ever added to any of the underlying base table(s) through the view.

Disadvantages of views

- **Update restriction:** In some cases, a view cannot be updated.
- **Structure restriction:** The structure of a view is determined at the time of its creation.

Example: If the defining query was of the form SELECT * FROM . . . , then the * refers to the columns of the base table present when the view is created. If columns are subsequently added to the base table, then these columns will not appear in the view unless the view is dropped and recreated.

- **Performance:** There is a performance penalty to be paid when using a view. In some cases, this will be negligible; in other cases, it may be more problematic.

Example: a view defined by a complex, multi-table query may take a long time to process as the view resolution must Join the tables together *every time the view is accessed*. View resolution requires additional computer resources.

View materialization

- **View materialization** stores the view as a temporary table in the database when the view is first queried.
- Thereafter, queries based on the materialized view can be much faster than recomputing the view each time.
- The speed difference may be critical in applications where the query rate is high and the views are complex, so that it is not practical to recompute the view for every query.
- **Benefits**: Integrity constraint checking and query optimization
- **Drawback**: Maintaining the currency of the view while the base table(s) are being updated.
- The process of updating a materialized view in response to changes to the underlying data is called **view maintenance**.

  -The basic aim of view maintenance is to apply only those changes necessary to the view to keep it current.

Any questions?